Progress Report 1: GraphWatch - Extracting Threat Intelligence From Telegra.ph

Alice "Allie" Roblee

Champlain College

CMIT-450-45: Senior Seminar Project - Fall 2026 (2026F7B)

Dr. Richard Pickering

November 1, 2026

Table of Contents

Introduction **3**
Project Plan & Weekly Timeline **4**
Work Completed **5**
API keys (#9) 5
Telegraph API client (#11) 5
Slugify (#60) 6
SQLite and the held-out set (#13) 6
Crowd check function (#12) 6
Provision VPS (#14) 7
Initial Crowd Check (#15) 7
Discussion (#8) 7
Findings that changed the plan 8
Digital Evidence **8**
Commit Log 9
Issue Board 9
Test Suite and Code Size 9
Crowd Result Table 10
Conclusion **11**
References **12**

PROGRESS REPORT #1 - GRAPHWATCH

Introduction

Telegraph (telegra[.]ph), Telegram's anonymous publishing platform, is ripe for abuse by threat actors. It requires no account, exposes a public API that requires no authentication, and uses a deterministic URL in the form of telegra[.]ph/TITLE-MONTH-DAY-COUNTER. While Telegram is one of the most closely monitored platforms in threat intelligence, Telegraph receives little attention even though Russia's FSB Center 18 is actively using it as a Dead Drop Resolver (DDR) in malware campaigns against Ukraine (Garçon & Sekoia TDR Team, 2026a–c; Security Service of Ukraine, 2021a–c). A dead drop resolver is a public web page that tells malware which Command and Control (C2) server to contact so the attacker can move the server without interacting with all of the infected machines (MITRE, n.d.).

The product of this project is GraphWatch, a Python tool being built in a public repository (github.com/alicerchamplain/CMIT450) that will produce an abuse feed from Telegraph's content. It will utilize Telegraph's API within its terms of service for data collection and then enrich that data using public threat intelligence feeds. GraphWatch will output threat intelligence indicators with metadata on when they were discovered and whether they are novel. When completed, GraphWatch will answer the research question I proposed: "Can a publishing platform's deterministic namespace, walked via its API, separate abuse from ordinary use for threat intelligence, and how far does that reach?" To answer this, I will measure Reach (the request cost of sweeping the platform for abuse candidates) and Novelty (the share of surfaced indicators absent from public threat intelligence feeds such as ThreatFox and OTX).

Project Plan & Weekly Timeline

The project plan proposed on October 24, 2026 is reproduced below.

Week	Proposed Tasks	Due	Status
1	Project proposal, set up GitHub project tracking.	Oct 25	Complete
2	Register accounts at ThreatFox and OTX. Build the wordlists, API client, and crowd check function. Run a crowd check for the past 30 days.	Nov 1	Complete. Explained in this progress report.
3	Write the sweep function with redaction. Sweep 30 days into a database. Read a sample and write initial scoring rules.	Nov 8	Planned
4	Write the classifier and scoring function. Score the first sweep, refine, configure daily sweeps.	Nov 15	Planned
5	Label sampled data for precision. Improve scoring; extend to the stretch goal if time permits. Write enrichment and score novelty.	Nov 22	Planned
Break	Daily sweeps continue running.	Nov 28	Planned
6	Assemble findings: reach, novelty, breadth. Draft the final report.	Dec 6	Planned
7	Finish the report and the demonstration video.	Dec 11	Planned

Week 2 Timeline - Issue numbers as tracked in the public GitHub repository

Issue	Planned Work	Estimate	Outcome
#8	Discussion: planning and resources.	2h	2.5h - Posted Oct 28, replied Oct 31 and Nov 1
#9	Register API keys: VirusTotal, AlienVault OTX, and Abuse.ch	0.5h	0.5h - Completed Oct 25
#11	Telegraph API Client	3.5h	4h - Completed Oct 25
#60	Slugify - Implement transliteration in GraphWatch for Cyrillic titles	1.5h	2h - Completed Oct 31

Issue	Planned Work	Estimate	Outcome
#13	SQLite Database Schema	3h	2h - Completed Oct 31
#12	Crowd Check Functionality	4h	5h - Completed Oct 31
#14	Provision VPS	2h	0.5h - Completed Oct 31
#15	Initial Crowd Check	1h	2h - Completed Nov 1
#16	Progress Report #1	1h	3h - This Report
#10	Word Lists Frozen	1h	0h - Moved to Week 3 Based on Findings
	Total Planned	19.5h	21.5h

Work Completed

API keys (#9)

AlienVault OTX, Abuse.ch (ThreatFox), and VirusTotal accounts registered. API keys have been generated and stored in a password manager.

Telegraph API client (#11)

API client written to graphwatch/telegraph.py. It builds URL paths from a title slug, month, day and counter. It calls the API's getPage endpoint, parses the API envelope response into FOUND, NOT_FOUND or FAILED and flattens a page's content tree into plain text and any links it contains. Additionally, the API client implements a rate limit, a request budget, automatic retries, and a host allowlist to prevent it from communicating with anything but telegra[.]ph. During implementation and testing I made some findings that required changing some of the design. First, Cyrillic titles get Latin paths by Telegraph's own custom transliteration scheme, which differs from a standard transliteration. For example, Ukrainian i is not transliterated at all and

passes through as a Cyrillic letter into their URL paths. Additionally, in Telegraph's URL path generator the first page of a day has no -1 suffix, and the counter suffix starts at -2. Finally, Telegraph's API responses return HTTP 200 (success) even when a page is not found instead of responding with a 404.

Slugify (#60)

The findings from #11 meant the word lists needed custom transliteration. A "slugify" function was written to graphwatch/slugify.py to transliterate title slugs. It uses a custom transliteration table as observed from Telegraph's dataset. Additionally, the word lists were rewritten to hold Cyrillic originals so that the tool does the transliteration instead of the wordlists having transliteration guesses.

SQLite and the held-out set (#13)

The SQLite schema was drafted in graphwatch/db.py and contains six tables: titles, pages, snapshots, events, http_log and crowd. A page row is created once and updated on every sighting. A snapshot row is written every fetch with a SHA-256 fingerprint of the title, description and content. The fingerprint excludes the view counter data to avoid changing the fingerprint from a view-count increase. Every content change is stored as a row in the events table. Every HTTP request is saved in http_log.

The 16 known dead drop resolver pages from public reporting were loaded as the held-out evaluation set. Of the 16, 15 are still live and were fetched and stored in the database.

Crowd check function (#12)

The crowd check function was written in graphwatch/crowd.py. It counts how many pages exist under one title on one calendar day without probing every counter value. It does this using a variation of the classic computer science algorithm called exponential search (Bentley & Yao,

1976). The crowd check works by doubling the counter until a probe misses, then bisects between the last hit and the first miss. This provides logarithmic search cost, meaning it can confirm 88,651 pages exist in only 52 API requests, saving significant time and resources compared to a linear search which would take at least 88,652 requests. The variation from the traditional algorithm is a gap check that probes two additional counters above any missed counter to avoid stopping early in the event a page was deleted.

Provision VPS (#14)

A DigitalOcean droplet running Ubuntu 24.04 and Python 3.12 was provisioned. The server was secured with SSH key authentication only and a firewall rule allowing only SSH from my residential IP address.

Initial Crowd Check (#15)

To orchestrate the crowd check run, I wrote a simple Command Line Interface (CLI) in graphwatch/cli.py. The initial project proposal called for 14 to 30 days. The 14-day run finished in under three hours, so it was rerun at 30 days. The 30-day crowd check counted every one of the 939 titles in the wordlists from October 3 to November 1. These first two crowd checks showed that the initial cap of 4096 on the counter as a safety limit was too low. Seven titles hit the initial limit. I then increased the cap to 2^20 and reran the crowd check for the 44 title-day combinations that hit the original cap.

Discussion (#8)

The initial discussion post was published Oct 28, with replies to classmates on Oct 31 and Nov 1.

Findings that changed the plan

- Telegraph does its own custom transliteration. This necessitated the creation of slugify.py to automatically transliterate the wordlists. Without it no Cyrillic title could have been counted.
- The evaluation set had a typo. One entry was written as mark-01-20-5 instead of Mark-01-20-5. While path lookups are case-insensitive via the API, the stored paths are not, so the first load of the evaluation set created a duplicate in the database. This was resolved by changing GraphWatch to store the API's path value instead of the string that was requested.
- Counters accumulate across years. Paths carry no year, so Links-10-22-11418 is the 11,418th page titled Links created on any October 22 since 2016. This necessitated changing the max crowd value from 4096 to 2^20.
- Initially, there was concern about hitting an undocumented rate limit as there is no published limit. The proposal assumed an API client capable of 8 requests per second but the initial client is limited to ~6.5 requests per second as it has no concurrency, meaning it has to wait for the previous API request to finish before making the next. This was partially done for simplicity's sake but also out of fear of hitting a rate limit. To date, no throttling or API errors have occurred. Concurrency will be added in Week 3.

Digital Evidence

The project proposal promised that every progress report would include at minimum the GitHub commit log and the crowd result table, scoring and evaluation output, generated results, and DDR findings once available.

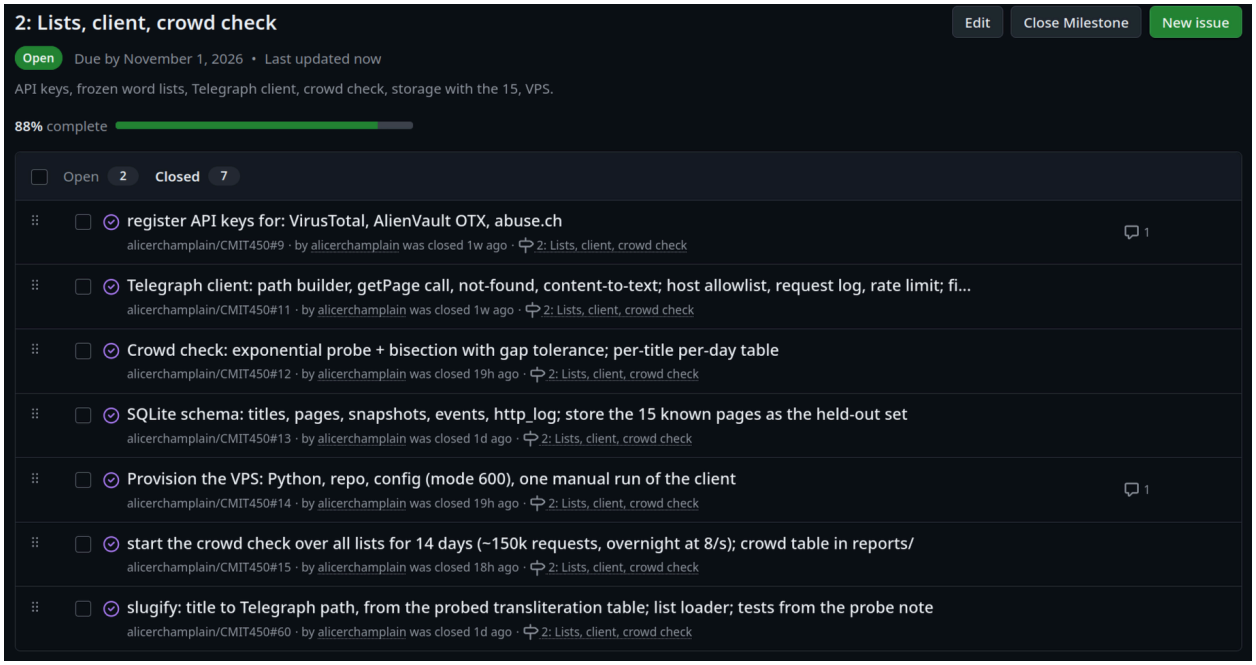
Commit Log

The following screenshot shows the commits that have been published to GitHub at github.com/alicerchamplain/CMIT450.

```
56c6d37 2026-11-01 First crowd check results. Closes #15
0e98833 2026-11-01 Keep the gap window inside max_n and raise the default cap to 2**20
31fb58b 2026-11-01 Rename ru-names to ru-words: the list mixes names and common nouns
4098cc9 2026-10-31 Add a basic initial cli for running the crowd check. Closes #15
cc2a960 2026-10-31 Add the crowd check: doubling probe with bisection and gap tolerance. Closes #12
a446390 2026-10-31 Store the held-out set; correct Mark-01-20-5 to the API's path. Closes #13
3f913ec 2026-10-31 Fix .gitignore line split
9d1eb84 2026-10-31 Finish SQLite schema and storage functions with tests. Related to #13
6b1d9d7 2026-10-31 Start building out db.py for database functions.
b9b9a0a 2026-10-31 Add slugify and word list loader from the probed transliteration table. Closes #60
6b8c871 2026-10-25 Update wordlists based on initial transliteration findings. Related to #10
9cba2d4 2026-10-25 Update API wrapper based on findings from some manual runs. #11
8173f5c 2026-10-25 Finish writing basic telegraph client class. Log initial API findings. Closes #11.
963d28a 2026-10-25 Add initial graphwatch package: Has telegraph path builder, envelope parser, content walkers
```

Issue Board

The following screenshot shows issues closed during Week 2 and is publicly available at github.com/alicerchamplain/CMIT450/milestone/2



Test Suite and Code Size

The following screenshot shows 255 tests passing with 3006 lines written in the codebase since the end of Week 1.

```
(.venv) allie@graphwatch:~/CMIT450$ pytest -q
.....................................................................................................
255 passed in 2.32s
(.venv) allie@graphwatch:~/CMIT450$ git diff --shortstat fddb7d2 HEAD -- graphwatch tests
 11 files changed, 3006 insertions(+)
```

Crowd Result Table

The following tables reproduce the crowd check results after the cap was increased to 2^20. The results were published to

github.com/alicerchamplain/CMIT450/blob/main/plan/reports/crowd-table-2026-11-01.md

Title-days counted	28,170 (939 titles × 30 days, October 3 to November 1)
Requests	207,001, all first attempt, all HTTP 200
Failures	0
Pages confirmed to exist	871,807
Pages per calendar day	29,060 average
Largest single title-day	Proverka-10-25, 88,651 pages
Latency	median 146 ms, p95 154 ms, max 1.7 s
Requests per title-day	7.3 average, 3 minimum, 56 maximum

Source List	Titles	With Pages	Zero	Pages	Requests
greetings	68	67	1	429,918	26,208
ru-words	416	404	12	259,346	91,292
brand-phishing	73	70	3	129,996	19,178
c2-tooling	79	65	14	12,642	16,360
crypto-theft	71	67	4	11,411	15,121
stealer-markets	53	48	5	10,749	10,194
trackers-droppers	56	52	4	9,381	12,011
proxy	58	42	16	5,796	9,616

Source List	Titles	With Pages	Zero	Pages	Requests
lures	65	19	46	2,568	7,021

30-day pages per title	Titles
0	105
1 to 9	158
10 to 99	199
100 to 999	430
1,000 to 9,999	40
10,000 to 99,999	5
100,000 or more	2

Conclusion

At the conclusion of Week 2 the project is going smoothly without any major issues. I originally planned to spend 19.5 hours on the project this week and spent about 21.5 hours. The majority of this time was spent on writing the actual code for GraphWatch. The findings from Week 2 changed my approach. The crowd numbers are cumulative since 2016, so the safety cap had to change. The title slugs use a custom transliteration dictionary, so a new slugify function was added. For Week 3 I will continue as planned and implement the sweep function with redaction. Additionally, I will sweep 30 days of content into a database and write initial scoring rules. The project is already pushing me to learn new Python and SQL functionality and Week 3 should impart some additional threat intelligence knowledge.

References

Bentley, J. L., & Yao, A. C.-C. (1976). An almost optimal algorithm for unbounded searching. *Information Processing Letters*, 5(3), 82–87. https://doi.org/10.1016/0020-0190(76)90071-5

Denissov, N. (n.d.). tgphind (Version 1.1.0) [Computer software]. GitHub. https://github.com/ndenissov/tgphind

Duncan, B. (2026, January 4). Cryptocurrency scam emails and web pages as we enter 2026 [Diary entry]. SANS Internet Storm Center. https://isc.sans.edu/diary/32594

Garçon, A.-J., & Sekoia TDR Team. (2026a, June 1). FSB's matryoshka #1/3: Gamaredon's gifts that keeps unpacking – GammaPhish and GammaWorm. Sekoia.io Blog. https://www.sekoia.com/blog/fsbs-matryoshka-1-3-gamaredons-gifts-that-keeps-unpacking-gammaphish-and-gammaworm

Garçon, A.-J., & Sekoia TDR Team. (2026b, June 3). FSB's matryoshka #2/3: Gamaredon's gifts that keeps unpacking – GammaLoad. Sekoia.io Blog. https://www.sekoia.com/blog/fsbs-matryoshka-2-3-gamaredons-gifts-that-keeps-unpacking-gammaload

Garçon, A.-J., & Sekoia TDR Team. (2026c, June 4). FSB's matryoshka #3/3: Gamaredon's gifts that keeps unpacking – GammaSteel. Sekoia.io Blog. https://www.sekoia.com/blog/fsbs-matryoshka-3-3-gamaredons-gifts-that-keeps-unpacking-gammasteel

Gen Digital. (2026). Inside the JDownloader supply chain attack. Gen Blogs. https://www.gendigital.com/blog/insights/research/inside-the-jdownloader-supply-chain-attack

INKY. (2022). Fresh phish: Phishers take advantage of Telegraph's loose governance. INKY. https://www.inky.com/en/blog/fresh-phish-phishers-take-advantage-of-telegraphs-loose-governance

Intel 471. (2024, December 16). Expanding source coverage: Adding Signal chats to threat intelligence. Intel 471 Blog. https://www.intel471.com/blog/expanding-source-coverage-adding-signal-chats-to-threat-intelligence

Microsoft Digital Security Unit & Microsoft Threat Intelligence. (2022, February 4). ACTINIUM targets Ukrainian organizations. Microsoft Security Blog. https://www.microsoft.com/en-us/security/blog/2022/02/04/actinium-targets-ukrainian-organizations/

MITRE. (n.d.). Web service: Dead drop resolver (T1102.001). MITRE ATT&CK. Retrieved October 24, 2026, from https://attack.mitre.org/techniques/T1102/001/

MITRE. (2026, October 1). Gamaredon Group (G0047, Version 3.4). MITRE ATT&CK. https://attack.mitre.org/groups/G0047/

Owda, A. (2026, August 19). How security teams track threat actor activity on Telegram without joining risky channels. SOCRadar Blog. https://socradar.io/blog/security-teams-track-threat-actor-telegram-channels/

Purbasari, A. (2016, November 23). Telegram launches Telegraph, an anonymous blogging platform. The Verge. https://www.theverge.com/2016/11/23/13728726/telegram-anonymous-blogging-platform-telegraph

Security Service of Ukraine. (2021a, November 4). SBU vstanovyla khakeriv FSB, yaki zdiisnyly ponad 5 tys. kiberatak – komentar kerivnyka DKIB SBU [SSU identified FSB hackers who carried out over 5,000 cyberattacks – comment by the head of the SSU Cybersecurity Department] [Video]. YouTube. https://www.youtube.com/watch?v=9CYArTPoijE

Security Service of Ukraine. (2021b, November 4). SSU intercepted conversations of FSB hackers who carried out over 5 000 cyberattacks on state bodies [Video]. YouTube. https://web.archive.org/web/20211104114113/https://www.youtube.com/watch?v=Rci5xiyMv7k

Security Service of Ukraine. (2021c, November 4). Tekhnichnyi zvit diialnosti Armahedon [Technical report on the activities of Armagedon]. https://ssu.gov.ua/uploads/files/DKIB/Технічний%20звіт%20діяльності%20Армагедон.pdf

Sublime Security. (n.d.). Link: Telegraph-hosted content [Detection rule]. Retrieved October 24, 2026, from https://sublime.security/feeds/core/detection-rules/link-telegraph-hosted-content-a1ccdfa2/

Telegram. (n.d.). Telegraph API. Telegraph. Retrieved October 24, 2026, from https://telegra.ph/api

Toulas, B. (2021, November 4). Ukraine links members of Gamaredon hacker group to Russian FSB. BleepingComputer. https://www.bleepingcomputer.com/news/security/ukraine-links-members-of-gamaredon-hacker-group-to-russian-fsb/

Trend Micro. (2026, June). Old WinRAR flaw fuels attacks on Ukraine: How unmanaged software keeps the door open. Trend Micro Research. https://www.trendmicro.com/en_us/research/26/f/old-winrar-flaw-fuels-attacks-on-ukraine.html

Vyas, A. (2023, July 28). Streamline your content with Telegram's Telegra.ph. Telegraph. https://telegra.ph/Streamline-Your-Content-with-Telegrams-Telegraph-07-28